CV1000 Blitter performance and behavior

Author: Buffi
Last updated: 2022-12-30
Document version: 1.04

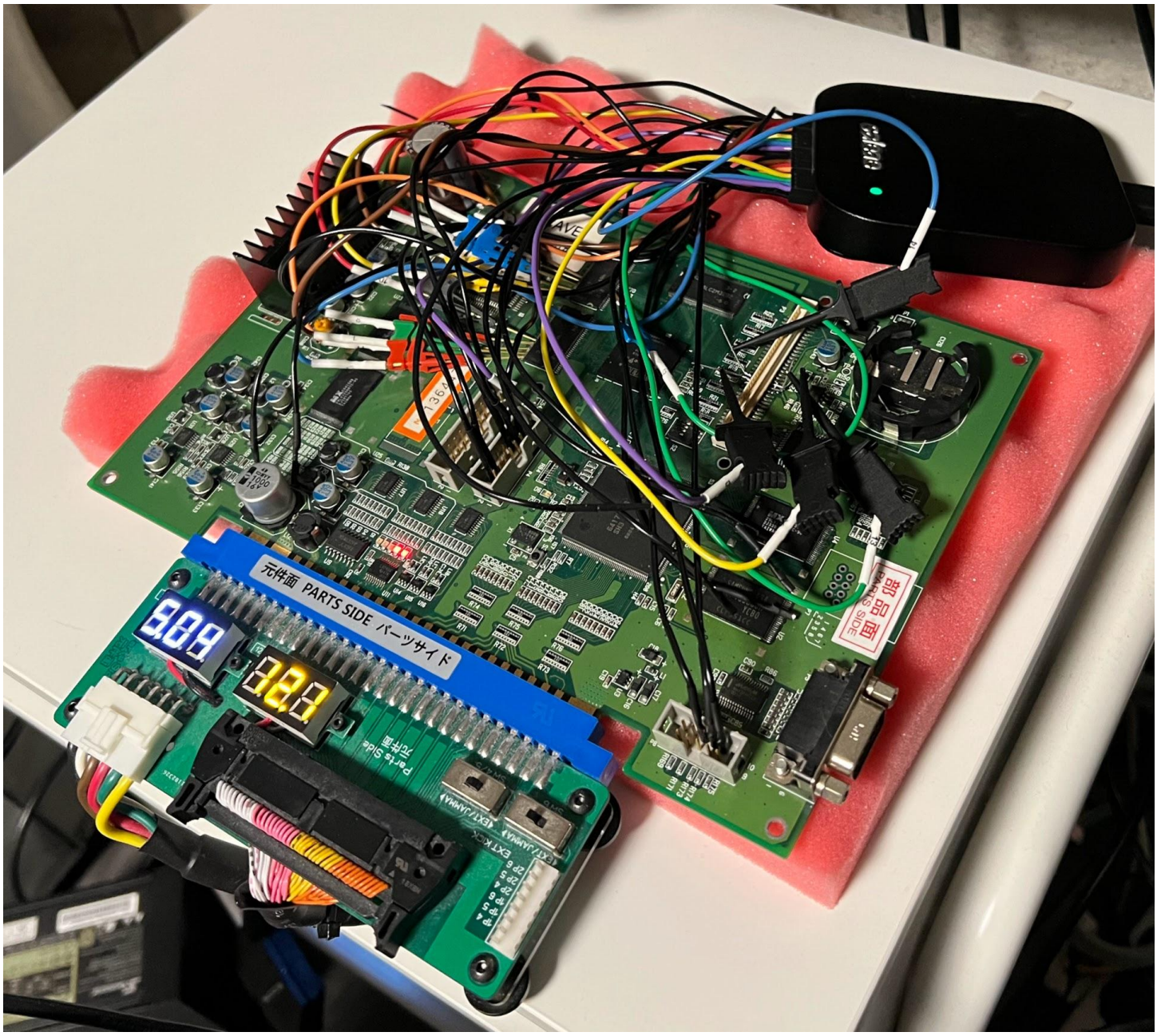


Table of contents

Scope and goal of document

This document aims to document behavior, performance and timing of the EP1C12 FPGA used as a Blitter on CV1000 PCB's. This research should make it possible to more accurately simulate the behavior of the Blitter in terms of slowdown.

It is not meant as a full reference for the Blitter and does not contain full documentation for all Blitter commands or operations.

Disclaimer on accuracy

Measurements were done by me (buffi) and there may be errors, mistakes and typos. I've done my best to describe my method of gathering data towards the end of the document, and welcome people replicating this setup and looking at it as well.

I have not fully reverse engineered the exact behavior for all types of scenarios, but this document should be sufficient to make a mostly accurate simulation.

CV1000 Blitter, and terminology used in this document

The "**Blitter**" as referenced in this document is the CV1000 Graphics Processor, implemented in an EP1C12 FPGA.

It can be addressed by "**Commands**" from the main CPU to perform actions or return state of the Blitter. The commands by themselves contain no information about graphics objects or drawing, but there is a command which sends a RAM address to the Blitter for fetching "Operations" and another command to start execution of these.

These "**Operations**" can either contain image data to be uploaded to VRAM, or instructions on how to draw them (copy them around in VRAM, possibly with modifications like transparency, blending and color levels).

Layout of Graphics data in VRAM

The VRAM for CV1000 consists of two 256Mb DDR SDRAM IC's. Each one of these has a bus width of 16 bits, and are addressed in tandem (CS, RAS, CAS, WE and A lines are tied together) for a 32 bit bus width and a total of 512Mb storage.

The pixel format used for graphics is ARGB1555 (16 bits per pixel).

The graphics memory is addressed by the blitter as a Bitmap with *width=8192* and *height=4096* pixels. This can be seen by looking at Draw and Upload commands sent to the Blitter, which will always fall within this space. This size also exactly adds up to the 512Mb of storage (*8192 * 4096 * 16b = 512Mb*).

The VRAM has its memory structured as:

- Total storage = 4 Banks
- 1 Bank = 8192 Rows
- 1 Row = 512 Columns of 32b each.

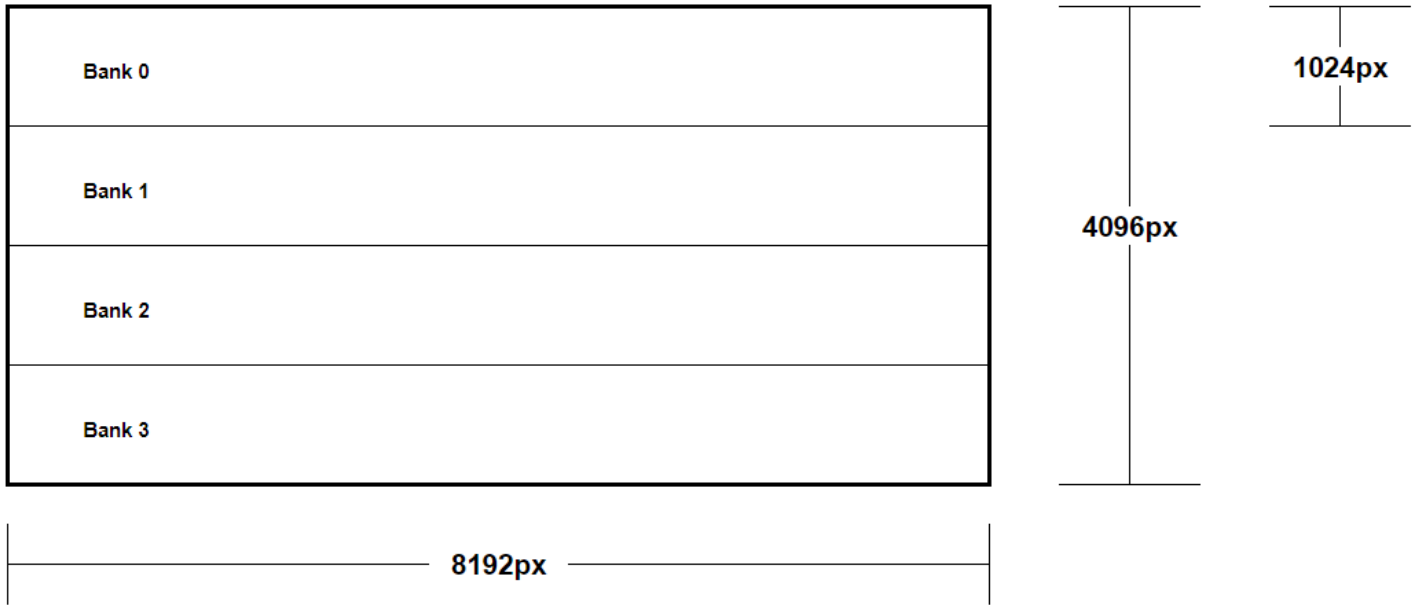
Each row is structured as a Bitmap of 32 x 32 pixels, which exactly fills up the Row memory:
`32 * 32 * 16b = 512 * 32b`

Since the bus width is 32b, each read/write of a Column will return/write two pixels at the time.

Each Bank consists of 8192 of these 32 x 32 Bitmaps, ordered left to right and wrapping at the 8192px width boundary, for a total size of *width=8192px*, *height=1024px* per Bank.

Visual representation of VRAM layout

The total area is split into four 1024px high Banks like this:



Each bank has 8192 Rows, each sized as 32x32 pixels, laid out like this:

Row X / Y in Bank	0	1	2	3	...	254	255
0	Row 0	Row 1	Row 2	Row 3	...	Row 254	Row 255
1	Row 256	Row 257	Row 258	Row 259	...	Row 510	Row 511
2	Row 512	Row 513	Row 514	Row 515	...	Row 766	Row 767
...	...	...	...	...	...	...	...
30	Row 7680	Row 7681	Row 7682	Row 7683	...	Row 7934	Row 7935
31	Row 7936	Row 7937	Row 7938	Row 7939	...	Row 8190	Row 8191

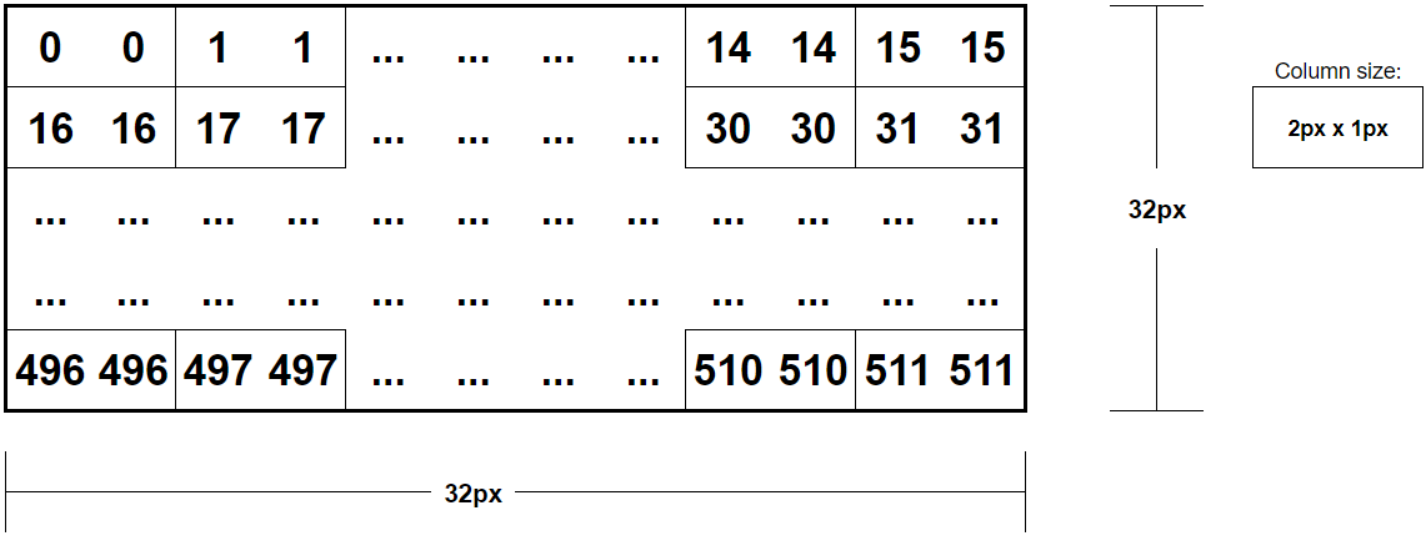
Row size:

32px x 32 px

1024px

8192px

As mentioned above, each Row contains 32x32 pixels. Since the VRAM bus width is 32 bits (2 pixels), this means that each Column in a Row will contain two pixels. The Column addresses for pixels in a Row look like this:



Pixel position to VRAM Bank, Row and Column

When reading or writing a pixel to position *(X, Y)*, the Bank, Row and Column can be identified using the following algorithms:

```
Bank = FLOOR( Y / 1024)
Row = FLOOR(X / 32 ) + 256 * FLOOR( (Y % 1024) / 32 )
Column = FLOOR( ( X % 32 ) / 2 ) + 16 * FLOOR( Y % 32 )
```

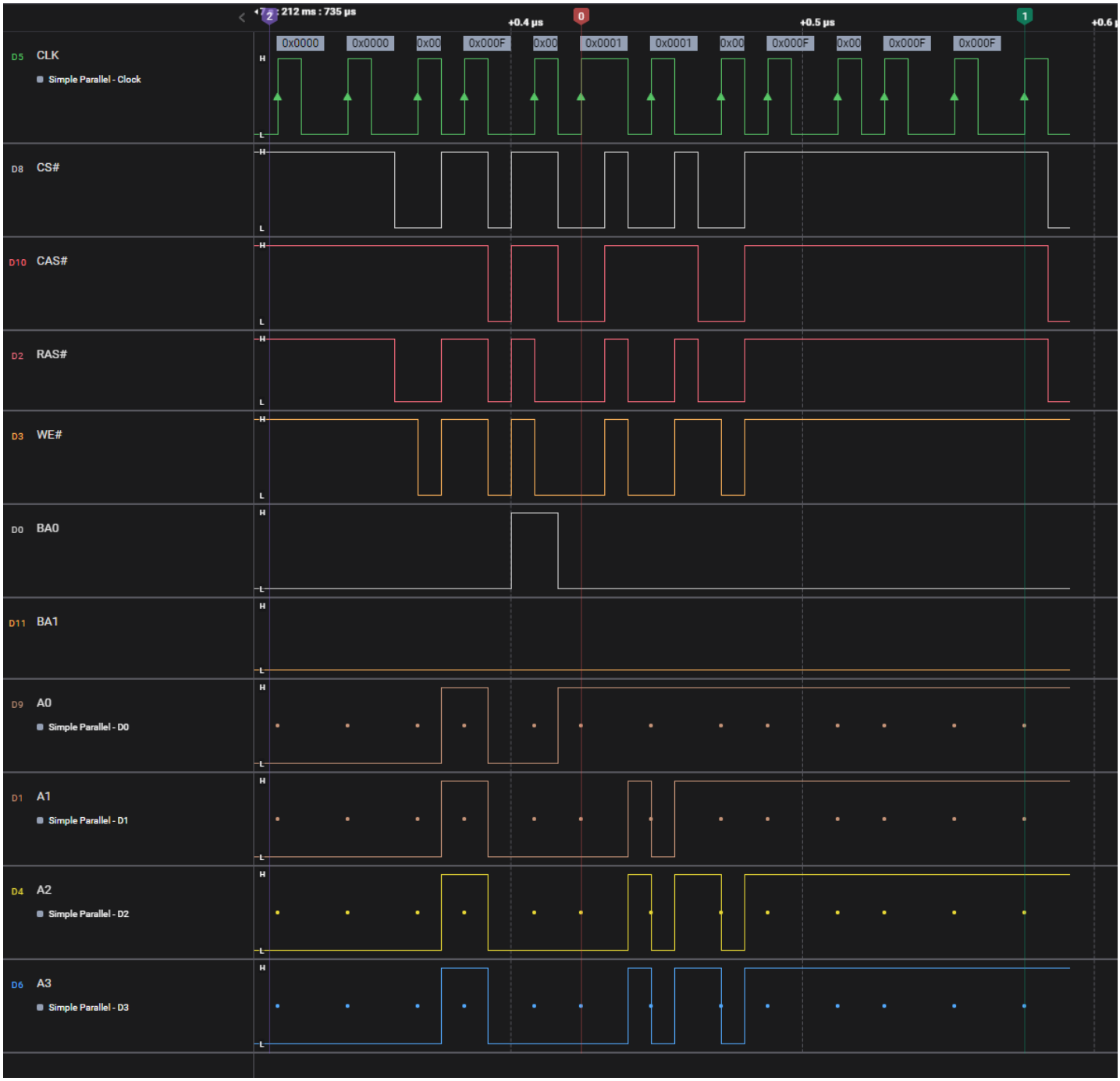
As an example, the first two pixels of image starting at position *X=2000, Y=3000* will be in *(Bank=2, Row=7486, Column=392)*.

For timing purposes, the exact Row addresses used aren't very important. The total count of Rows read from/written to does however have implications to timing, since switching Rows has overhead in terms of clock cycles needed.

Reading/Writing Graphics data

Writes are always done four pixels at the time, to offsets evenly divisible by four

The Blitter configures VRAM to run with “**Burst Length**” set to 2, which means that it will always do batches of two reads/writes at once. Since each individual read/write handles two pixels, this means that a minimum of four pixels are handled each time VRAM is accessed. The image below shows the command to set Burst Length to 2 (LOAD MODE REGISTER with low four bits set to 0x1).



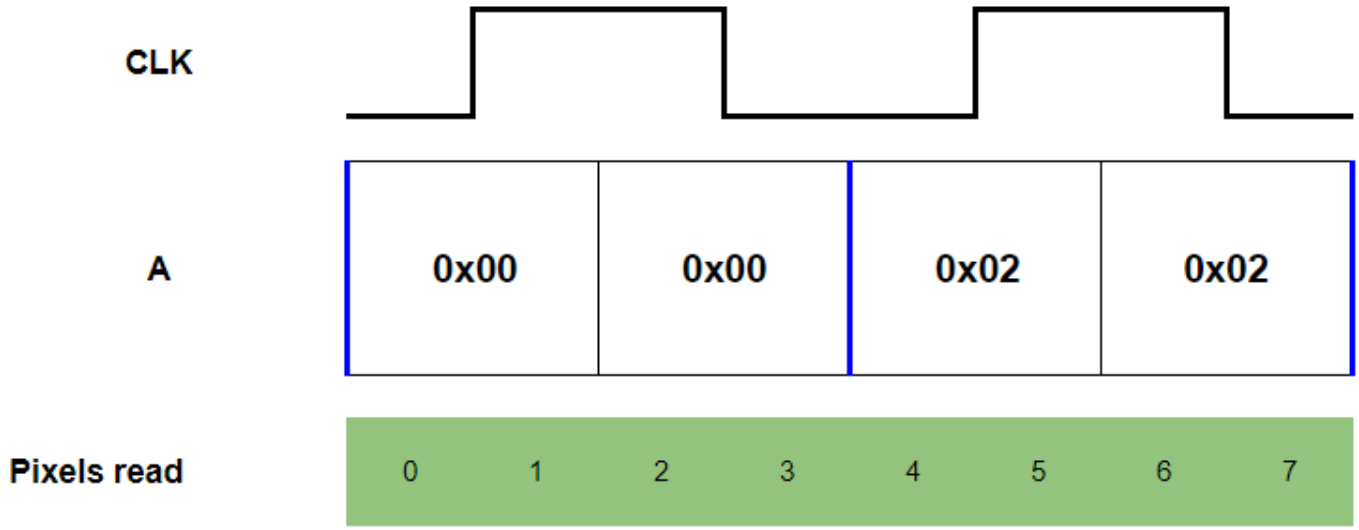
Since all reads and writes to the VRAM are made to pixel X positions evenly divisible by four. This means that A0 will always be low for VRAM operations from the Blitter.

If both the X position and the size of the image is evenly divisible by four, there are no wasted read/write cycles. For other operations, there will be wasted cycles due to operations on unchanged pixels. As an example, if drawing 8 horizontal pixels to the position (X=1, Y=0) the Blitter will do the following:

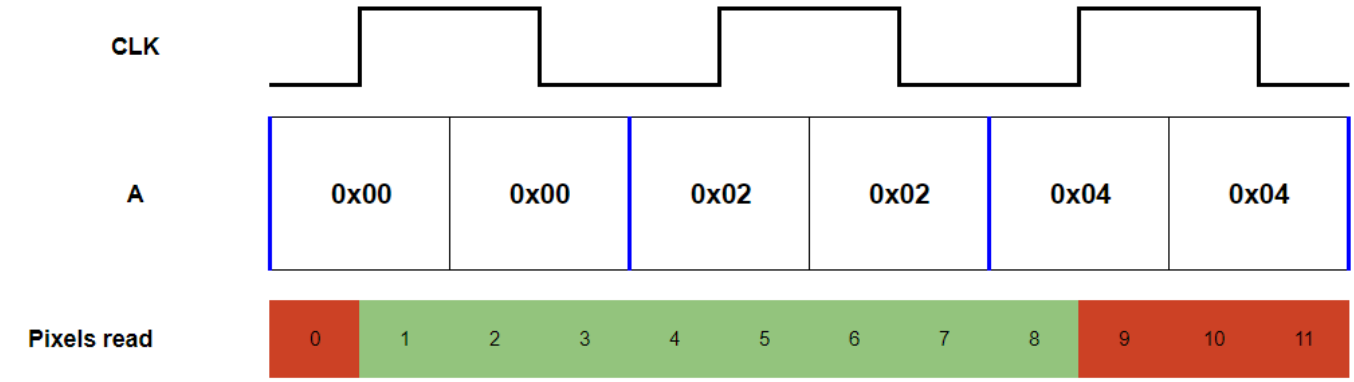
- Read the data to draw (This has always been Uploaded to positions evenly divisible by 4)
- Read the target area to draw to. This means first reading the four pixels X=0 to X=3 and then reading the pixels X=4 to X=7, since the Blitter will only read four pixels at the time at offsets divisible by four.
- Merge the data to draw with the data of the target area
- Write the merged data back, which will also require writing the area between X=0 and X=7.

This means that Draws to offsets not evenly divisible by 4 will always add 4 pixels (1 CLK) of overhead to the read and write of the target area.

This first example shows a write of a 8 pixel wide image to X=0. Since this is perfectly aligned with four-pixel boundaries, there's no additional overhead. Since Burst Length is 2, the same offset is used for reading four pixels at the time. Note that reads are executed on both rising and falling edges of CLK, since this is DDR RAM.



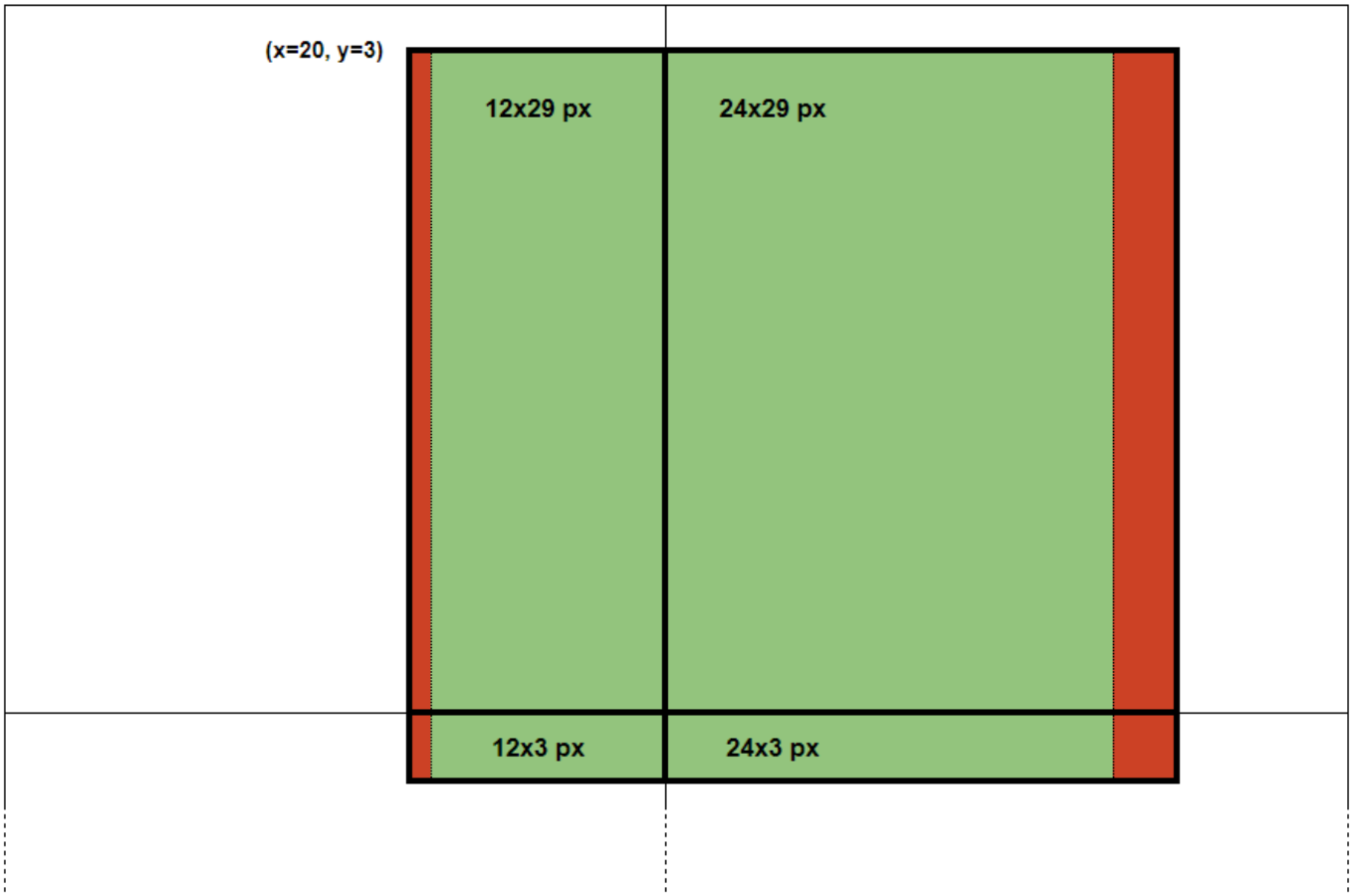
This second example shows a write of a 8 pixel wide image to X=1. This requires reading and writing data both before and after the actual target pixels, to get the write aligned to four-pixel boundaries. The positions marked as red are overhead.



Writing across multiple 32x32 pixel VRAM Rows

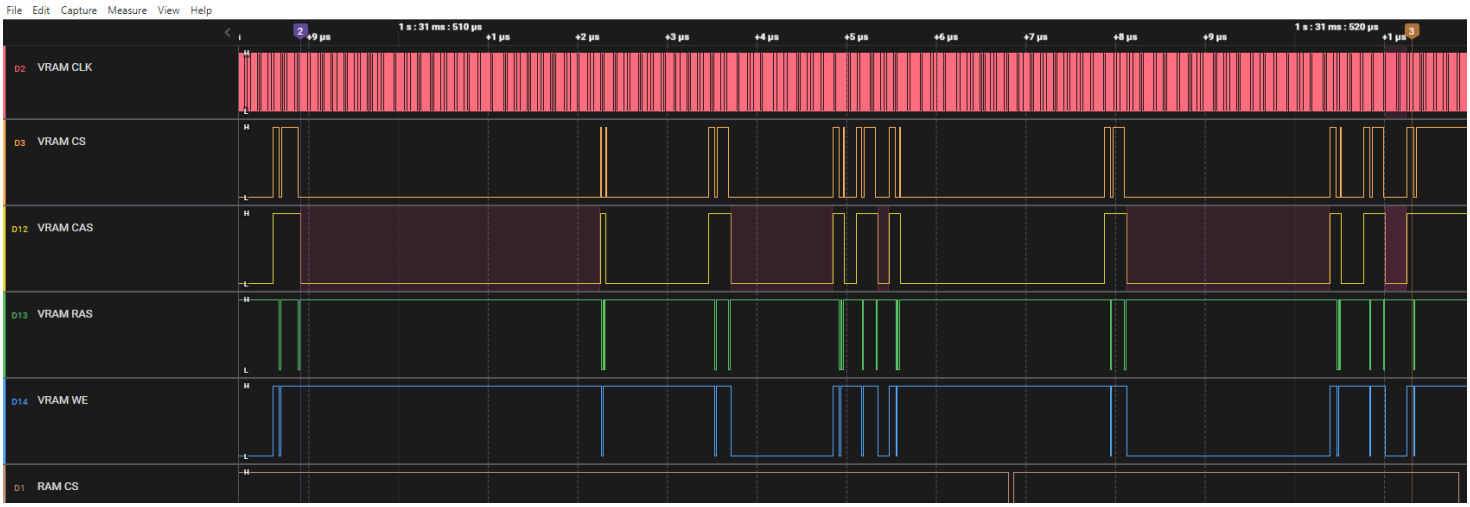
As mentioned in “Layout of Graphics data in VRAM”, the VRAM is split into 32x32 pixel areas. If Drawing to a single area, it can be done in one continuous sequence of writes, but if a sprite is drawn across multiple areas, the VRAM needs additional overhead for addresses to switch Row.

As an example, let’s take an instruction which Draws a 32x32 sprite starting at (X=21, Y=3). This crosses four different areas and has an X-offset not evenly divisible by four (which means having to write 36x32 as described in section above).



This can be seen in practice below where five segments are marked red on the CAS line. These are:

- 256 CLK Read (1024 pixels)
    - *32x32 Sprite, efficiently stored in a VRAM Row*
- 87 CLK Write (348 pixels)
    - *12x29 px Bitmap*
- 9 CLK Write (36 pixels)
    - *12x3 px Bitmap*
- 174 CLK Write (696 pixels)
    - *24x29 px Bitmap*
- 18 CLK Write (72 pixels)
    - *24x3 px Bitmap*



The overhead for switching to a new Row is due to having to issue a “*Precharge*” followed by a “*Active*” command to the RAM (markers 4 an 5 in image below).

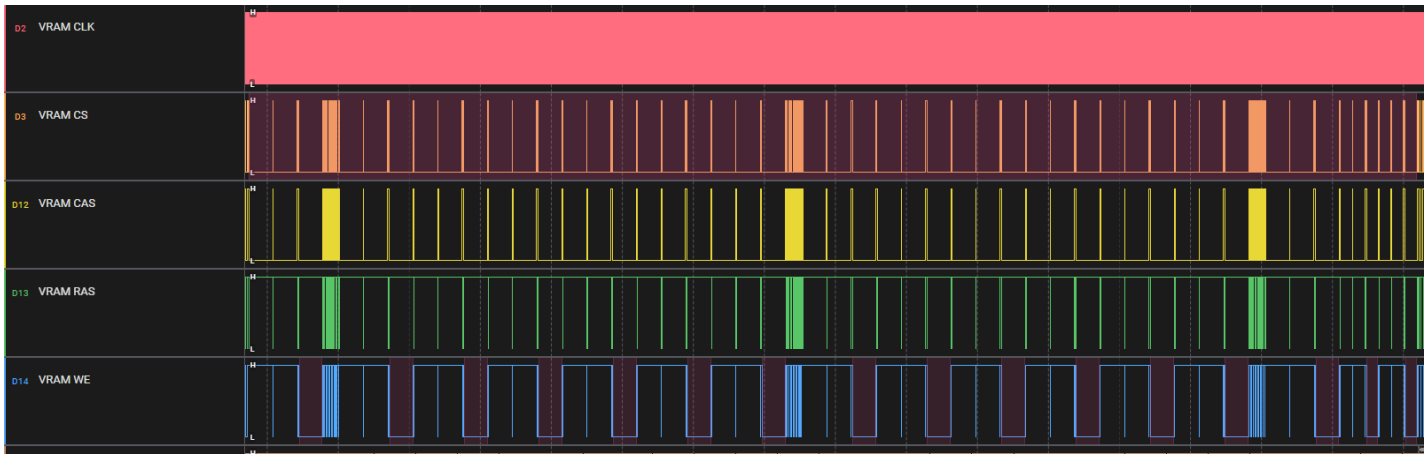
CLK cycles needed as overhead will depend on the types of operations switched between (read/write). The Draw command documents this further down in the document.

A separate example is available below where a large 240x64px sprite is Drawn to *(x=768, y=0)*. This sprite is aligned to four-pixel boundaries. It is however very large, spanning 7.5 32x32 Areas horizontally, and 2 areas vertically like this:

(x=768, y=0)

32x32 px	32x32 px	32x32 px	32x32 px	32x32 px	32x32 px	32x32 px	16x32 px
32x32 px	32x32 px	32x32 px	32x32 px	32x32 px	32x32 px	32x32 px	16x32 px

This can be seen in practice below where there are 14 larger writes of full Rows. Then the 16x32px bitmaps are the two thinner WE segments on the far right.
Also note that since this image is large, there are three horizontal line reads happening as well. This is covered in the section "**Rendering output video latency**" later in this doc.

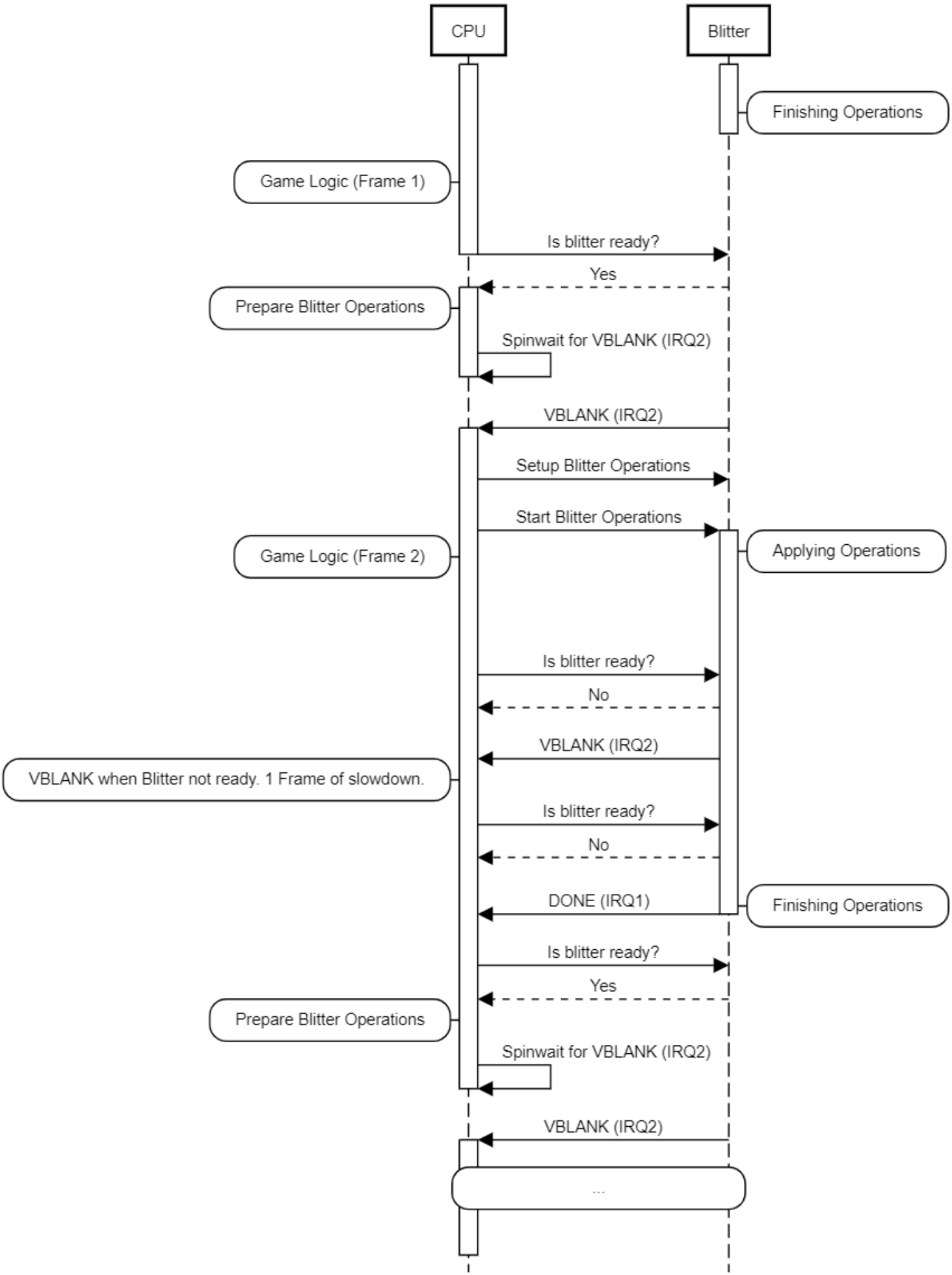


Overall sequence of Blittering

The sequencing of CPU interacting with the Blitter looks something like this:

- Game logic
- CPU checks if Blitter is still executing operations
    - Read from *0x18000010* until *0x00000010* is returned
- When Blitter is ready, the CPU lays out the operations that the Blitter should perform in SRAM.
- CPU spin-waits for an IRQ2 interrupt
    - For some scenarios, the CPU will wait for multiple interrupts, to introduce artificial slowdown. This is unrelated to the Blitter delays, (handled by program ROM).
- Blitter notifies CPU of VBLANK by interrupting on IRQ2
- CPU signals the address of operations
    - Writes the current Operation list address to *0x18000008*
- CPU signals the Blitter to start work
    - Write *0x00000001* to *0x18000004*
- CPU swaps Operation memory location for the next frame to a new address.
- Until all operations are executed, the Blitter repeats the following sequence parallel to CPU execution:
    - Blitter pulls the BREQ pin on CPU low
    - CPU ACKs this by pulling BACK low. This tells the Blitter to start reading operations from memory
    - Blitter reads 64 bytes from SRAM.
    - Blitter signals to the CPU that it's done by pulling BREQ high.
    - Blitter starts executing instructions (see separate section below).
    - Once instructions have been executed, repeat.
- At the end of the sequence, the Blitter signals to the CPU that all computations are done by asserting IRQ1.

The sequence Diagram below illustrates one frame where Blitter processing has finished executing at the time of game logic being done, followed by a frame where a VBLANK interrupt occurs before the Blitter is ready (causing one frame of slowdown).


CPU
Blitter
Finishing Operations
Game Logic (Frame 1)
Is blitter ready?
Yes
Prepare Blitter Operations
Spinwait for VBLANK (IRQ2)
VBLANK (IRQ2)
Setup Blitter Operations
Start Blitter Operations
Game Logic (Frame 2)
Applying Operations
Is blitter ready?
No
VBLANK (IRQ2)
VBLANK when Blitter not ready. 1 Frame of slowdown.
Is blitter ready?
No
DONE (IRQ1)
Finishing Operations
Is blitter ready?
Yes
Prepare Blitter Operations
Spinwait for VBLANK (IRQ2)
VBLANK (IRQ2)
...

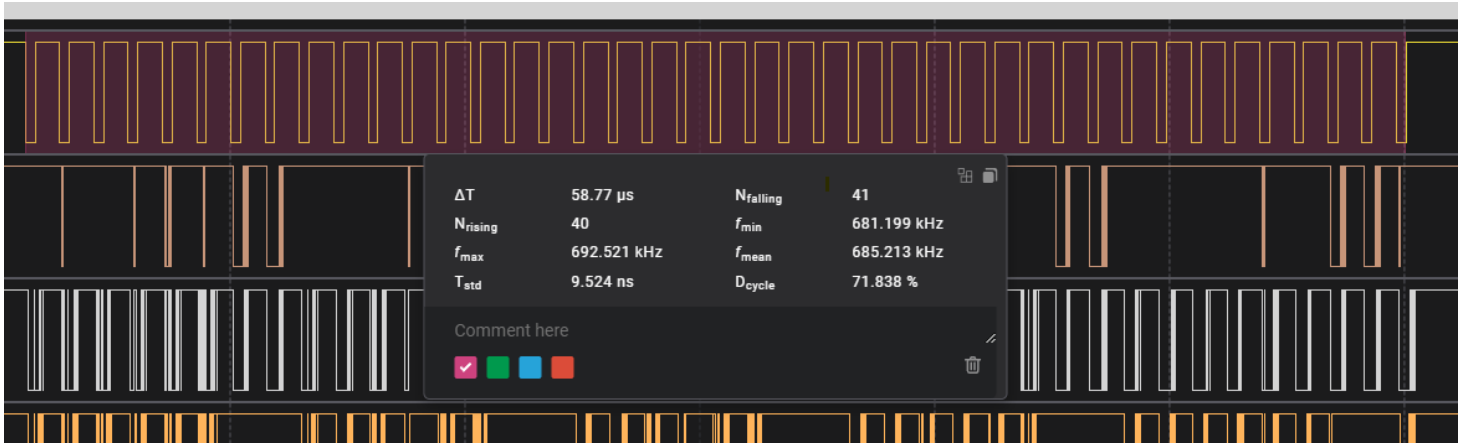
Blitter operations

Blitter has four types of instructions, **Upload**, **Draw**, **Clip** and **Exit** (names from MAME are used for consistency).

Upload

- Contains 16 byte fixed header followed by *WIDTH * HEIGHT * 2* bytes of pixel data
- Blitter will read pixel data as 64 byte chunks from SRAM (16 SRAM CLK Pulses).
- Takes *(WIDTH * HEIGHT * 2 + 16) / 4* RAM CLK pulses to finish executing
- Data is inserted into VRAM directly. This does not add much extra delay on Blitter since it's done concurrently to reading it from SRAM.
- Since the payload will typically span more than a single 64-byte block, in addition to the RAM access, there is additional delay waiting for the next BREQ/BACK cycle.
- Time to execute
    - RAM CLK is 50Mhz (measured by me), so one pulse is 20ns.
    - Between each 64 Byte/16 CLK sequence, there will be "gap" delays imposed by the FPGA having to request control of SRAM again.
      This usually takes approx 1.13us each cycle time. There's some variance here depending on how quickly the CPU acks the BREQ request.
    - Example 1: 8x8 image upload encountered at offset 5 in a (16 CLK) read cycle.
        - Needs *(8*8*2+16)/4 = 36* RAM CLK pulses
        - This will be spread out over four 16-CLK read cycles, which means two "gap" delays.
            - 7 CLK in the current cycle
            - 16 CLK in the next cycle
            - 13 CLK in the third cycle
        - Total time will be: *20ns * 36 + 1.13us * 2 = 2.98us*
    - Example 2: 256x5 image upload encountered at offset 0
        - Needs *(256*5*2+16)/4 = 644* RAM CLK pulses
        - This will be span 41 16-CLK read cycles, causing 40 "gaps"
        - Total time will be: *20ns * 644 + 1.13us * 40 = 58.08us*

As mentioned, there's some variance in this, but here's an example from a 256x5 Draw command on a Pink Sweets PCB. 58.77us is pretty close to the 58.08us estimate.



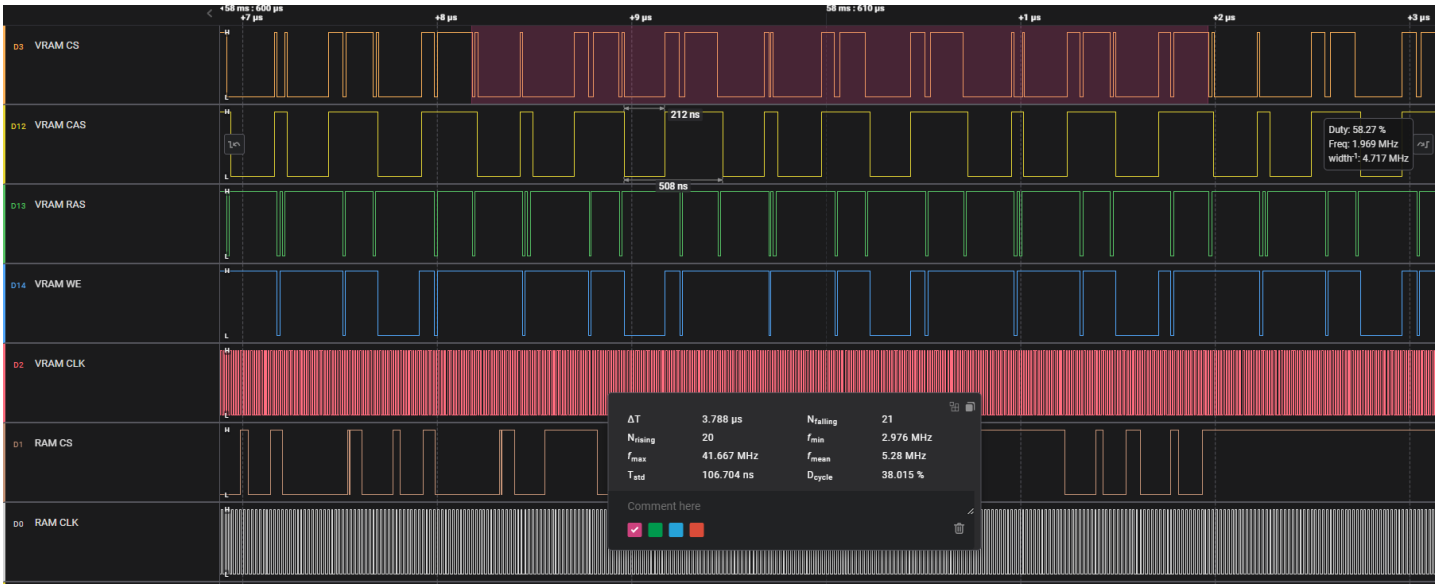
Draw

- CLK pulses mentioned for this is on VRAM which is 76.8MHz (13ns CLK pulses)
- Contains a 20 byte message, containing source and target information.
- Typically, the Blitter will read three of these each time it fetches data from RAM, since 64 bytes are transmitted each time. These are then queued for work by the Blitter.
- Latency will vary with the numbers of pixels to draw (called NUM_PIXELS) below.
- Sequence of a Draw is:
    - Blitter reads X source pixels to draw
    - Blitter reads X pixels from target location to draw too (needed for blending/transparency, but always read)
    - Blitter writes X pixels to target location
- If the Sprite target location only spawns one Row in VRAM, this is done once. Otherwise, the writes are split up as described in “Reading/Writing Graphics data”.
- Each VRAM CLK pulse will read/write 32 bits on both rising and falling edges (since memory is DDR3). This means that 64bits (4 pixels) will be read/written per pulse.
- There’s some addressing overhead for the different operations:
    - Switching Row from Read to Read: 5 CLK
    - Switching Row from Read to Write: 20 CLK
    - Switching Row from Write to Read: 10 CLK
    - Switching to drawing a new sprite: 10 CLK (In addition to Write-to-Read overhead)
- The reading of commands from SRAM is done concurrently to execution of Blitter commands already queued, so it’s safe to ignore that latency (unlike for uploads where they are coupled).

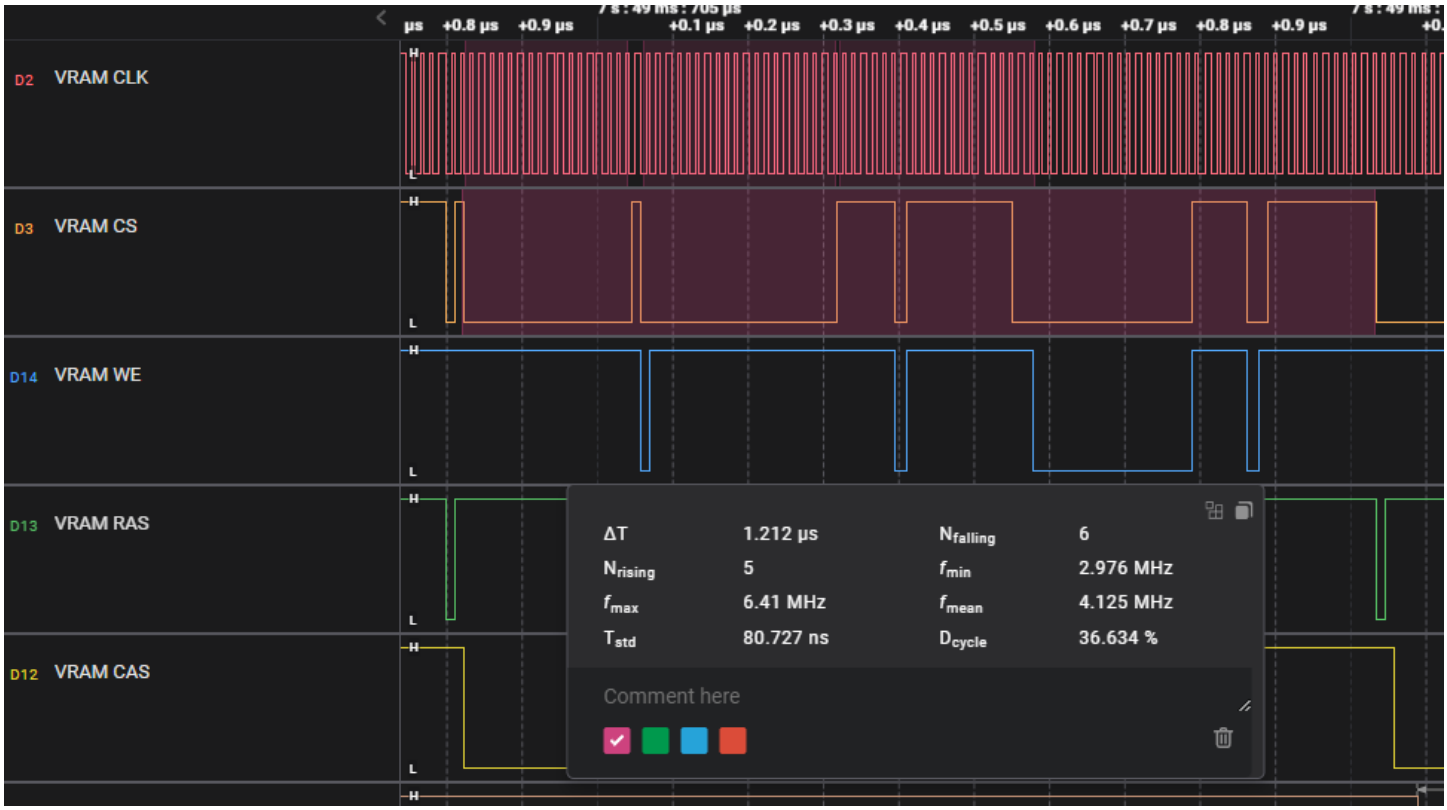
Some examples are below.

Drawing 8x8 sprites

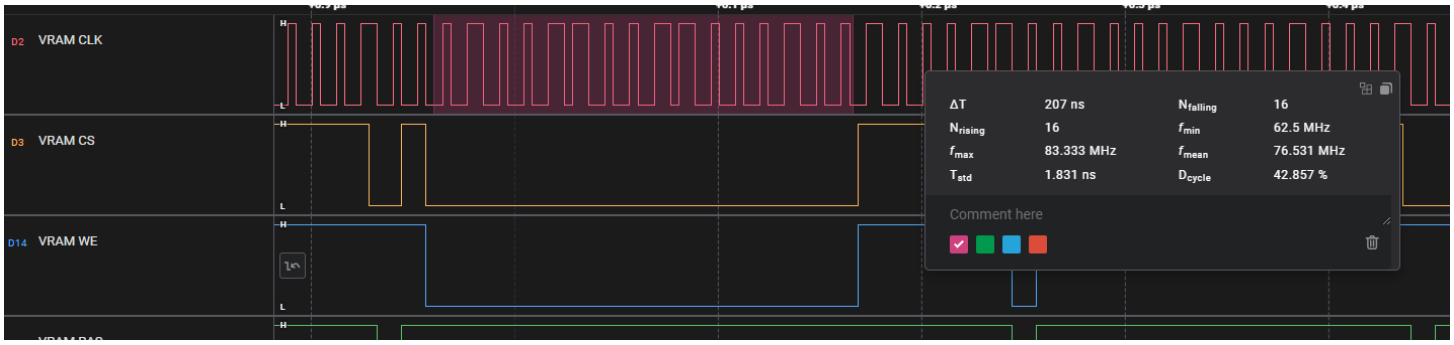
8x8 sized sprites seem very common.
Here’s a zoomed-out view showing three such writes, taking about 3.8us total.
Each write has VRAM CS go low three 16 CLK durations. The first two are reads (VRAM WE = 1). The third one is Write (VRAM WE = 0).



Here's a single write zoomed in. The Read and Write durations are clearly visible. The shorter durations of VRAM CS being pulled low when switching between Read and Write (and back) are related to address switches in the RAM.



Here's even more zoomed in on the write to VRAM.



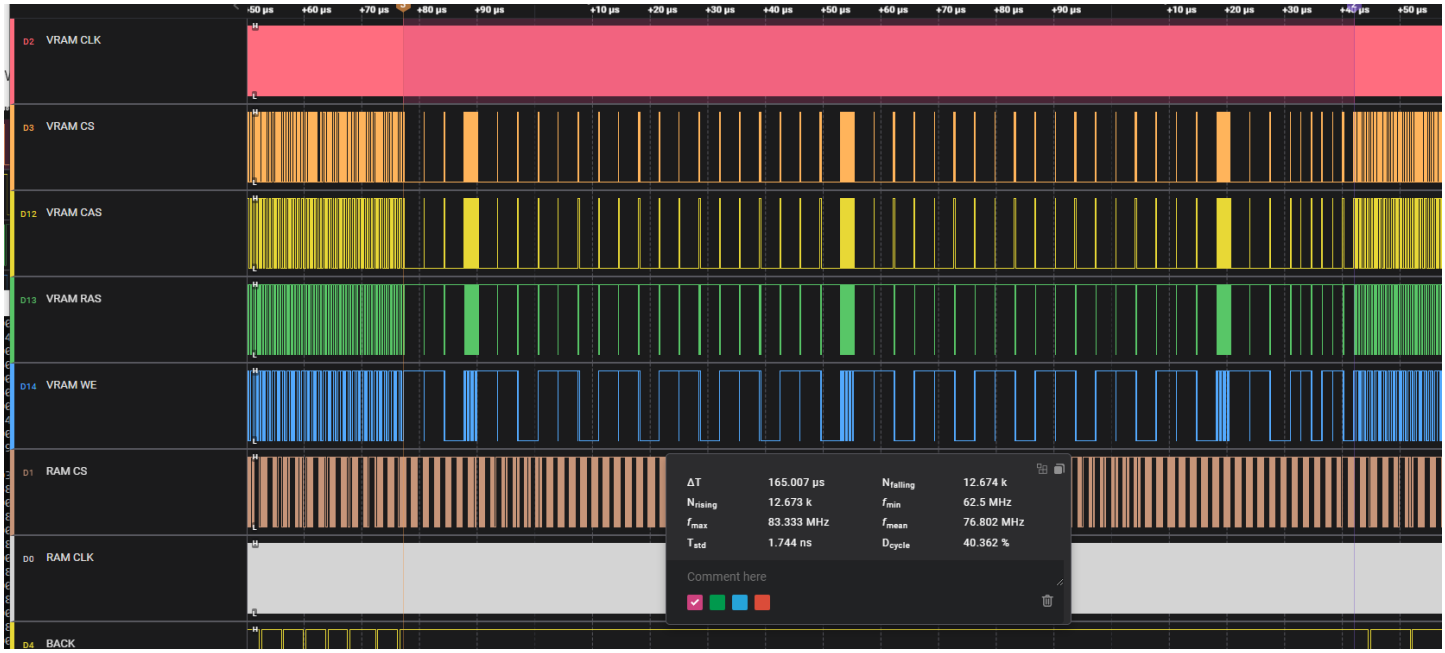
As we can see in the second image. The actual time for this write was about 1.212us (including the overhead of switching operations).
The estimate CLK needed above the images would end up very close:

- 16 (Read 64 pixels source)
- 5 (Read-to-read switch)
- 16 (Read 64 pixels destination)
- 20 (Read-to-write switch)
- 16 (Write 64 pixels destination)
- 10 (Write-to-read switch)
- 10 (Switch to next sprite)
- **Total = 93 CLK. 93*13ns = 1.209us**

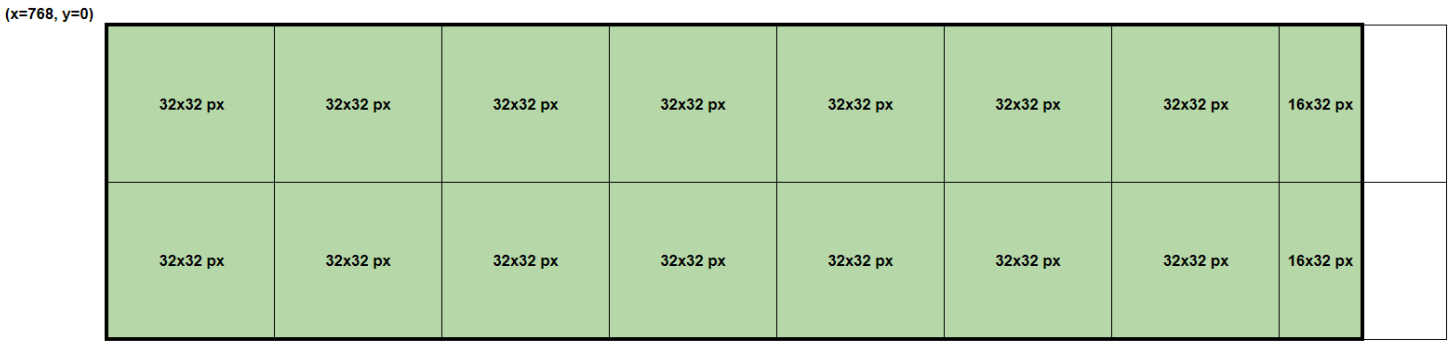
Drawing 240x64 sprite

Larger sprites will take long enough to draw that they will not have time to be drawn between two horizontal scanlines, which will affect the latency.
This image below shows a 240x64 being drawn. Since this is 15360 pixels, the FPGA will fetch quite a lot of data, and the full duration is about 165us.

The three thick lines in the middle of the draw are horizontal line reads for display purposes (See section “Rendering output video latency”.



This image will write to 16 Rows (32x32 px areas) in VRAM where 14 will be filled completely, and two will be only half drawn.



Time in VRAM CLK for one VRAM Row of 32x32 pixels will be:

- 256 (Read 1024 pixels source)
- 5 (Read-to-read switch)
- 256 (Read 1024 pixels destination)
- 20 (Read-to-write switch)
- 256 (Write 1024 pixels destination)
- 10 (Write-to-read switch)
- **Total = 803 CLK. 803*13ns = 10.439us**

Using the same calculations, the CLK needed for the two half-Row writes will be 419 CLK = 5.447us. Then the 10 CLK for switching to the next Sprite will be added on top of that.

This means that the full write should be:

```
14 * 803 CLK + 2 * 419 CLK + 10 CLK = 12090 CLK (157,170us)
```

This is significantly smaller than the 165us measured in practice. That's due to the three horizontal-line pulses taking up 2.16us each.

`157.170us + 3 * 2.16us = 163.65us` which is close to the measurement!

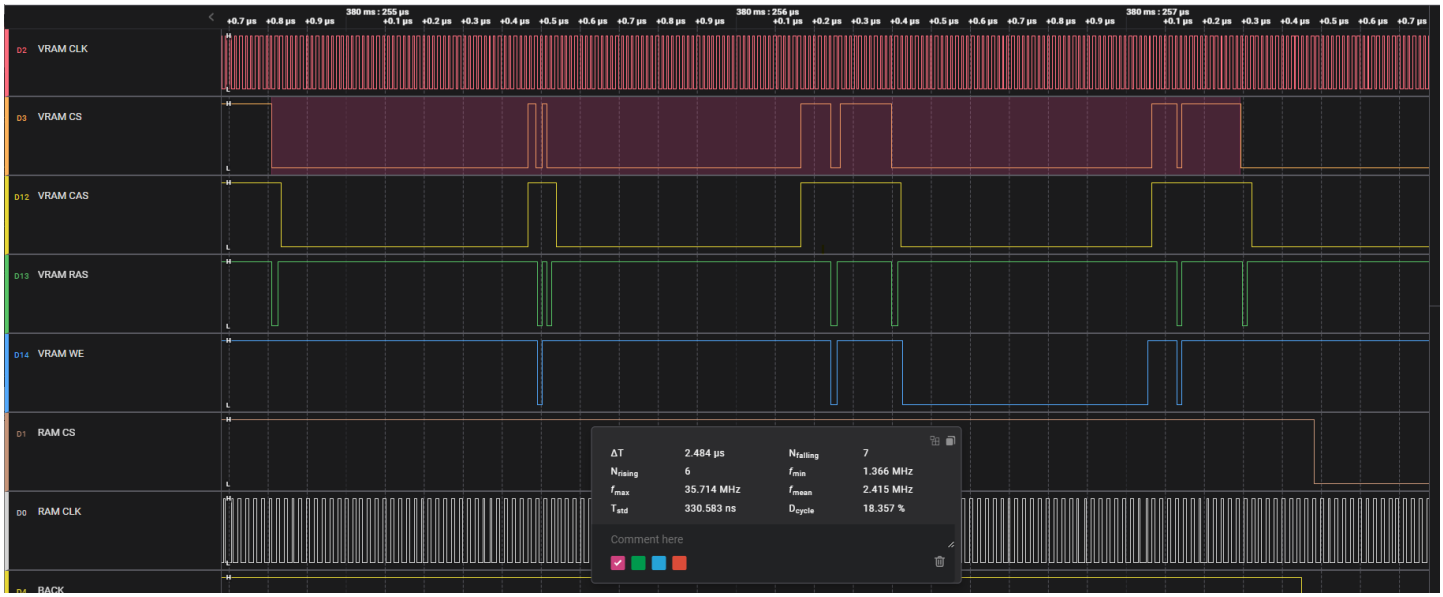
Drawing 16x12 sprites

Similar to 8x8 sprites, but 192 pixels means 48 CLK is needed to read/write everything.
The example below assumes both read and writes only span on VRAM Row each.

Time estimate will be:

- 48 (Read 192 pixels source)
- 5 (Read-to-read switch)
- 48 (Read 192 pixels destination)
- 20 (Write-to-read switch)
- 48 (Write 192 pixels destination)
- 10 (Write-to-read switch)
- 10 (Switch to next sprite)
- **Total = 189 CLK. 187*13ns = 2.457**

Measured latency is 2.484us, so fits very well.

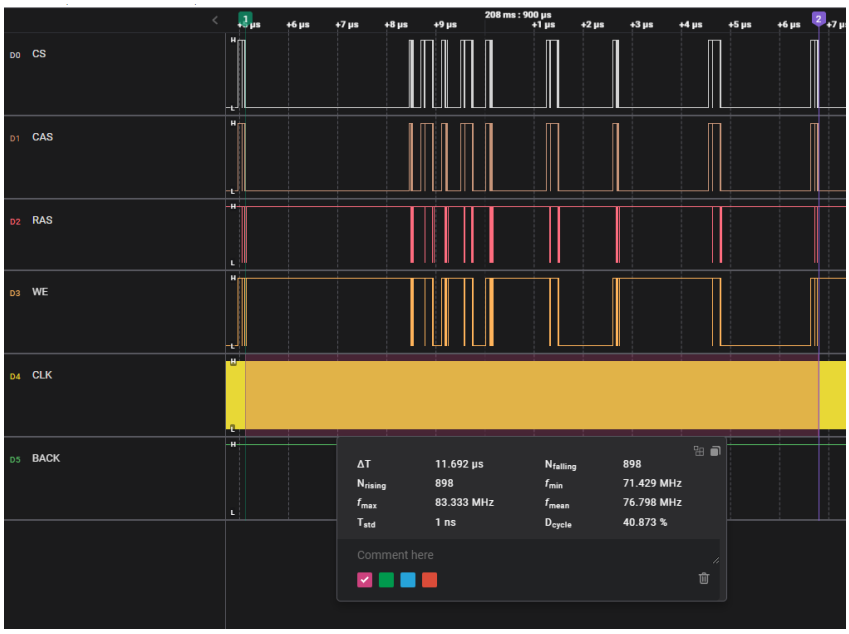


Blending and Transparency

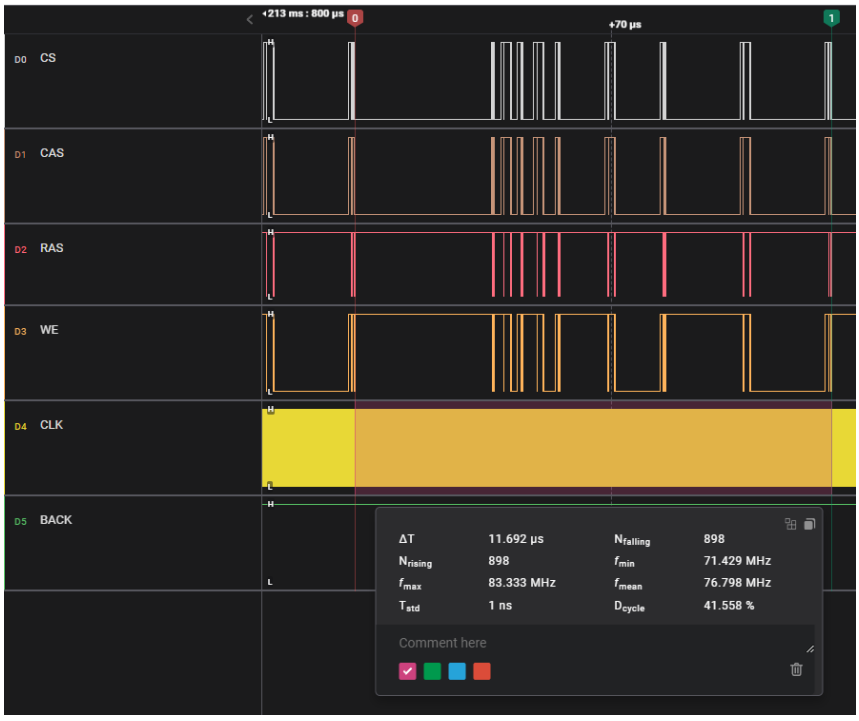
Having Blending or Transparency enabled does not modify the time it takes to Draw a bitmap in any ways that I can see.

I verified this by using the Object Test in the Special Mode test menu of Muchi Muchi Pork. Enabling Transparency and/or Blending does not change anything in terms of timings, see images below:

This first image shows a 32x32 part of a larger Sprite being drawn without any Transparency or Blending.



This second image shows a 32x32 part of the same sprite, being drawn with Transparency and Blending enabled (A and T options switched to enabled in the Test Menu). Timing is identical.



Clip

- Contains a 4 byte message, stating a clipping area for drawing sprites.
- Does not seem to cause any additional latency directly.
- Maybe this reduces the draw time for images clipped? This needs more investigation (see ”Future work” section)

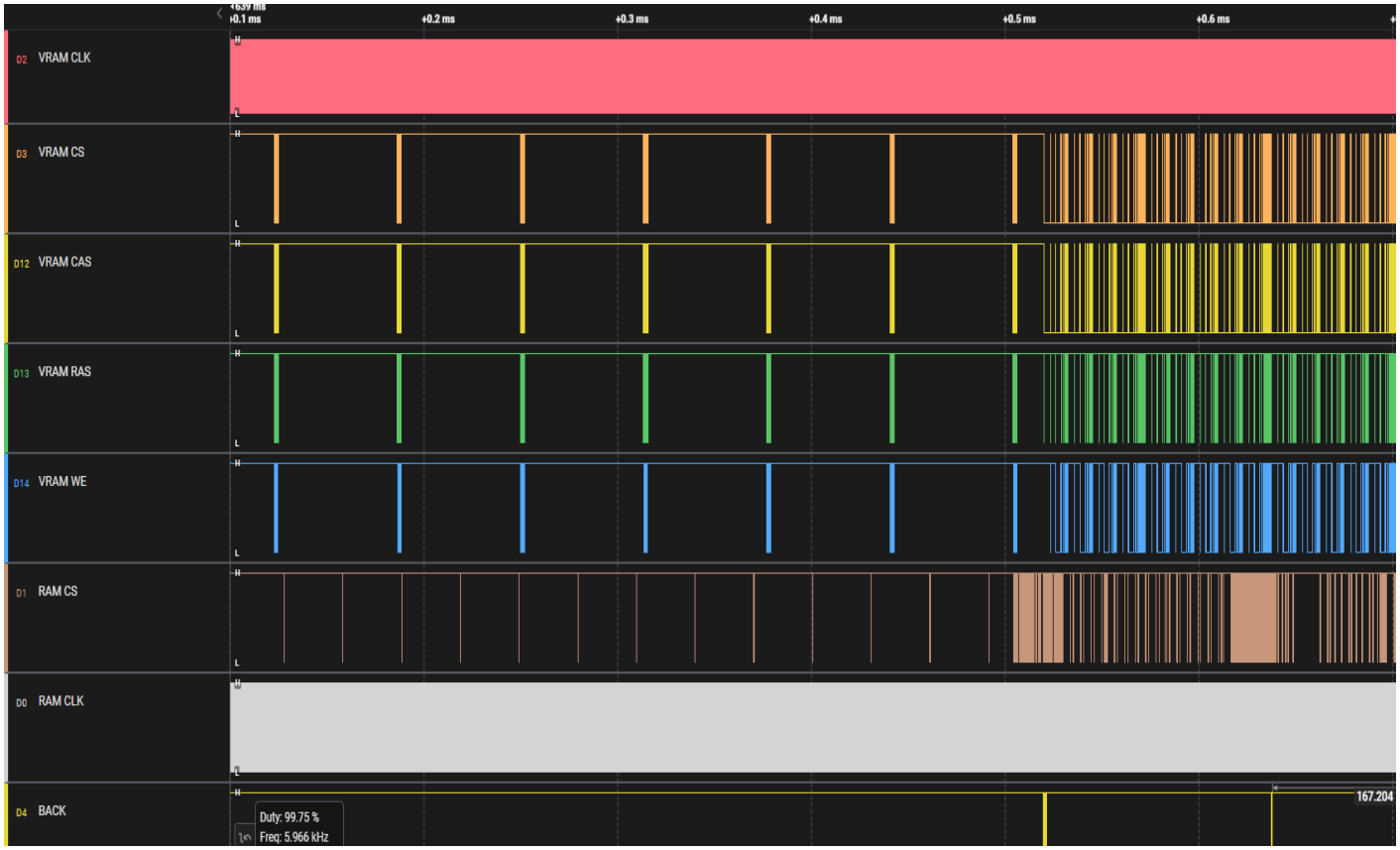
Exit

- Contains a 4 byte message
- Signals to the Blitter that this is the end of the list.
- Does not cause any additional latency.

Rendering output video latency

Every 63.6us the Blitter will read a line of graphics data, which will block it from executing Blitter instructions. This takes 2.16u each time.

In the image below, notice the periodic VRAM events prior to Blitter execution starts. These will block Operations from triggering when they occur during Blitter execution.



Formulas for Blitter latency

These formulas describe the calculations above more concisely.
I'm sure there's some stuff I'm missing, but these calculations should be sufficient to provide good approximations for Blitter performance.

Uploading images

```
NUM_RAM_CLK = (NUM_PIXELS * 2 + 16) / 4
Time needed = NUM_RAM_CLK * 20ns + FLOOR( NUM_RAM_CLK / 16) * 1.13us
```

Drawing images

```
VRAM_ROW_HEIGHT = CEIL( ( (IMAGE_Y % 32) + IMAGE_HEIGHT ) / 32 )
VRAM_ROW_WIDTH = CEIL( ( (IMAGE_X % 32) + IMAGE_WIDTH ) / 32 )
NUM_VRAM_ROWS = VRAM_ROW_HEIGHT * VRAM_ROW_WIDTH
NUM_VRAM_CLK = (NUM_PIXELS * 3 / 4) + NUM_VRAM_ROWS * (5 + 20 + 10) + 10
Time Needed = NUM_VRAM_CLK * 13ns
```

Total time when including periodic graphics output delays:
Given a list of instructions, with total time "INSTRUCTION_TIMES"

```
HLINE_DELAY = FLOOR_USEC( SUM ( INSTRUCTION_TIMES ) / 63.6u ) * 2.16u
Total time needed = SUM( INSTRUCTION_TIMES ) + HLINE_DELAY
```

Unknown behavior

Difference between Blitter versions

Earlier CV1000 releases will have a different Bitstream transmitted to the FPGA compared to later releases. These early titles often got patches later which used the new Bitstream, which indicates that either something was optimized or at least bugs fixed in it.

It's not clear what the difference is between the different versions, but it could be interesting to investigate.

Method of gathering data

A PCB running Pink Sweets was used for all measurements in this doc, unless noted otherwise.

Data was gathered using a Saleae Logic Pro 16 logic analyzer and Electro PJP Microprobes.

The ROM's of the board were dumped and used in the MAME debugger, to compare signals sent in the emulated logic vs signals seen on-board.

One way of identifying Blitter Operations in the logic analyzer output is to:

- Find a screen with mostly static sprite sizes. Examples are anything in the test menu (especially the Special Mode Test menus in the Yagawa games are useful) and title screens.
- Run: *wpset 18000008,1,w,1,{dump blitter_ops.txt,(wpdata-0xA0000000),20000}*
- Parse blitter_ops.txt with this script and compare to logic analyzer data for the same screen on pcb.

To debug Blitter behavior, useful points to probe are:

- SH3 BREQ/BACK
    - BREQ is pulled low when FPGA wants access to SRAM
    - BACK is pulled low by the SH3 when this access is granted
    - You don't really need both.
- SH3 D31 - D28
    - Looking at the highest 4 bits of the DATA bus allows identifying which Operations the Blitter reads from SRAM. This is aligned with the CLK of SRAM during the reads from Blitter. Mostly it's useful for verifying that you are seeing Draw Operations.
- SH3 CS6
    - Pulled low when SH3 sends commands to Blitter. These are routed through the CPLD at U13, and works as CS for Blitter commands.
- VRAM CLK (Note: Both VRAM use same CLK)
- VRAM CS# (Note: Both VRAM use same CS#)
    - Low when VRAM is active
- VRAM WE#, CAS#, RAS# (Note: Both VRAM have these shared too).
    - These three together signal what type of operation is performed on VRAM
- VRAM A
    - Needed to see where VRAM operations are written/read
- SRAM CS
    - Low when SRAM is used
- SRAM CLK

Future work

Future investigations on Blitter performance will be tracked on my Github project for CV1000 research.

Some future stuff I plan to look at there is:

- Impact of Clip operation to Draws being ”clipped” at edges in terms of timing.
- Difference between different Blitter versions
- Double-checking all VRAM addressing overheads, and documenting them more in-depth. It appears that the current results are very slightly below measured values, so something might be off by a few CLK.

If you have suggestions on things to check or want sample logic analyzer outputs, feel free to reach out to me on Github or the Arcade-Projects forums.

Thanks to

rtw: Has been very helpful with insights on CV1000 hardware, suggestions for how to proceed with this research as well as reviewing this document.

Olifante: Helped me out by analyzing the IRQ behavior for VBLANK and the “Blitter has finished execution” interrupts, which allowed me to track down how it works on PCB. Also has been very helpful in terms of helping me understand the CV1000 program flows in general.